

Crivo

Manual técnico do projeto

Editor de pixel art baseado em matrizes, em JavaScript, com camadas, animação, histórico ilimitado, paletas clássicas e exportação para jogos. API em Django com PostgreSQL e ferramenta de linha de comando em Java.

Copyright (©) 2026 Erik Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 5
- Para que serve 5
- O que o Crivo faz 5
- Tecnologias 6
- Identidade visual 6

Como rodar 7
- Requisitos 7
- Primeira execução com Docker 7
- Sem Docker, com Python 7
- Só o editor 8

Conhecendo a tela 9
- Menus 10

Ferramentas de desenho 11
- Simetria 11

Visualização 13

Cores e paletas 14
- Cor principal e secundária 14
- Paletas predefinidas 14
- Suas paletas 14
- Importar e exportar paletas 14
- Gerador de tons 15

Camadas e animação 16
- Camadas 16
- Quadros 16
- Papel cebola 16

Tamanho da matriz e transformações 18

Importação e exportação 19
- Exportar imagem 19
- Sprite sheet 19

Exportar dados da matriz 20
Importar imagens 21
O arquivo .crivo 22

Salvando o trabalho 23
Servidor ou navegador 23
Salvar, versões e conflitos 23
Rascunho automático 24

Por dentro: matemática e estruturas de dados 25
A matriz de pixels 25
Composição das camadas 25
Linhas: algoritmo de Bresenham 25
Elipses: ponto médio na caixa 25
Balde de tinta: preenchimento por varredura 26
RLE com varint 26
Histórico ilimitado 26
Cores em OKLab e OKLCH 26
Escala por vizinho mais próximo 27

Arquitetura do código 28
Módulos do editor 28
Fluxo de eventos 28
Servidor Django 29

API 30
Autenticação 30
Rotas 30
Códigos de resposta 31
Exemplos 31

Ferramenta em Java 32

Configuração 33
Antes de publicar na internet 33

Testes e integração contínua 35

Personalizando o Crivo 36

Uma paleta fixa nova 36
Uma ferramenta nova 36
Um formato de exportação novo 36
Ícones, cores e atalhos 36

Atalhos do teclado 37

Solução de problemas 39

Capítulo 1

Visão geral

O Crivo é um editor de pixel art de código aberto que roda no navegador. Ele trata a imagem como ela realmente é: uma matriz de pixels, cada um com quatro canais (vermelho, verde, azul e transparência). Todas as ferramentas leem e escrevem direto nessa matriz, e a tela só mostra o resultado ampliado, sem suavização.

O projeto foi feito para ser baixado, usado, estudado e adaptado. O editor funciona sozinho no navegador; com o servidor Django ligado, os projetos e as paletas passam a ficar num banco de dados, com histórico de versões e uma API para outros programas.

Para que serve

Para desenhar sprites, ícones, itens, personagens, cenários em blocos e animações curtas, e levar o resultado direto para um jogo. Além de PNG e sprite sheets, o Crivo exporta a própria matriz em JSON, CSV, JavaScript, Python, C e Java, para quem escreve o jogo do zero e quer os pixels como dados.

Também serve como material de estudo de computação gráfica de baixo nível: o código implementa, sem bibliotecas, a manipulação de `ImageData`, o algoritmo de linha de Bresenham, a elipse de ponto médio, o preenchimento por varredura, a composição alfa, a codificação RLE e a conversão de cores para o espaço OKLab.

O que o Crivo faz

Desenho

- Lápis e borracha com pincel de 1 a 16 pixels, quadrado ou redondo.
- Balde de tinta com 4 ou 8 vizinhos, tolerância de cor, modo contíguo ou global e leitura da camada atual ou de todas.
- Linha, retângulo e elipse, vazados ou preenchidos, com Shift para 45°, quadrado e círculo perfeitos, e Alt para desenhar a partir do centro.
- Conta-gotas, e Alt + clique para pegar uma cor com qualquer ferramenta de desenho.
- Simetria horizontal, vertical ou nos dois eixos, valendo para todas as ferramentas.

Organização

- Camadas com visibilidade, opacidade, bloqueio, duplicação, reordenação e mescla.
- Quadros de animação com duplicação, reordenação, FPS, papel cebola e reprodução.
- Histórico de desfazer e refazer sem limite de passos, guardando só o que mudou.
- Matriz de 1×1 até 1024×1024, com âncora para centralizar ou expandir, escala por vizinho mais próximo, giro de 90° e espelhamento.

Cores, importação e exportação

- Paletas Crivo, Game Boy, NES, Pico-8, CGA, CGA modo 4, Commodore 64 e 1 bit, além das suas, que você cria, importa e exporta.

- Gerador de tons em OKLCH com desvio de matiz.
- PNG e JPEG em tamanho real ou ampliados; sprite sheet horizontal, vertical ou em grade, com JSON das posições.
- Dados da matriz em JSON, CSV, JavaScript, Python, C e Java.
- Importação de imagens com detecção do tamanho do pixel e fatiamento de sprite sheets.

Tecnologias

Camada	Tecnologia	Papel
Editor	JavaScript (módulos ES), Canvas 2D, HTML5, CSS3	Toda a interface e o núcleo de desenho, sem bibliotecas externas
Testes do editor	Node.js 20+ (`node:test`)	22 testes do núcleo: cores, geometria, balde, RLE, documento, histórico, formatos e exportação
Servidor	Python 3.13, Django 6.1, Gunicorn, WhiteNoise	API REST, validação, versões, paletas e entrega do editor
Banco	PostgreSQL 17 (ou SQLite)	Projetos, camadas, quadros, células, versões e paletas
Linha de comando	Java 21 (só o JDK)	Informações, paleta, PNG, sprite sheet e matriz em Java a partir de um `.crivo`
Execução	Docker Compose	PostgreSQL e aplicação com um comando
Integração contínua	GitHub Actions	Testes em JavaScript, Django com PostgreSQL e Java, e construção da imagem Docker

Identidade visual

O nome vem do crivo, a peneira de malha quadriculada: a mesma grade que organiza a pixel art. O logo é um ponteiro de mouse em pixels, em preto sobre branco, o primeiro desenho que quase todo mundo já viu feito pixel a pixel.

A interface usa a paleta do projeto, também disponível como paleta de desenho:

Vinho	**Ardósia**	**Café**	**Linho**	**Oliva**	**Rosé**	**Palha**
#5D1C16	#A3B4BE	#5C4034	#E6DDCE	#7A6E46	#D6BDB8	#F6E4A6

No tema claro, o linho é o fundo, o vinho marca a ferramenta ativa e os botões principais, e a ardósia é a área em volta da matriz. No tema escuro, a palha assume o papel de destaque.

Capítulo 2

Como rodar

Requisitos

Jeito de rodar	Precisa de	O que você ganha
Docker (recomendado)	Docker Desktop ou Docker Engine com Compose	Editor, API e PostgreSQL prontos, com versões dos projetos
Python	Python 3.12 ou mais novo	Editor e API com SQLite, sem instalar banco
Só o editor	Node.js 20 ou mais novo (ou qualquer servidor de arquivos)	O editor completo, salvando no navegador
Linha de comando	JDK 21 ou mais novo	Converter arquivos `.crivo` em PNG, sprite sheet e código Java

Primeira execução com Docker

1. Baixe o repositório (botão **Code → Download ZIP** no GitHub, ou `git clone`).
2. No Windows, dê dois cliques em `iniciar.cmd`. No Linux ou macOS, rode `./iniciar.sh` no terminal, dentro da pasta.
3. Na primeira vez, o script copia `.env.example` para `.env` e troca a senha do banco e a chave secreta do Django por valores aleatórios.
4. O Docker baixa a imagem do PostgreSQL, constrói a imagem do Crivo e sobe os dois serviços. As migrações rodam sozinhas e criam as paletas predefinidas.
5. Quando `/api/v1/saude` responde, o navegador abre em **http://localhost:8080**.

```
./iniciar.sh
docker compose logs -f web
docker compose down
docker compose down -v
```

O segundo comando acompanha o registro da aplicação, o terceiro desliga tudo mantendo os dados, e o último apaga também o volume do banco.

Sem Docker, com Python

```
python -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

No Windows, ative o ambiente com `.venv\Scripts\activate`. Sem nenhuma variável definida, o Django roda em modo de desenvolvimento e grava no arquivo `crivo.sqlite3`, na raiz do projeto. O editor abre em **http://localhost:8000**.

Para usar um PostgreSQL já instalado, defina `CRIVO_DB_ENGINE=postgresql` e as variáveis `CRIVO_DB_HOST`, `CRIVO_DB_PORT`, `CRIVO_DB_NAME`, `CRIVO_DB_USER` e `CRIVO_DB_PASSWORD` antes do `migrate`.

Só o editor

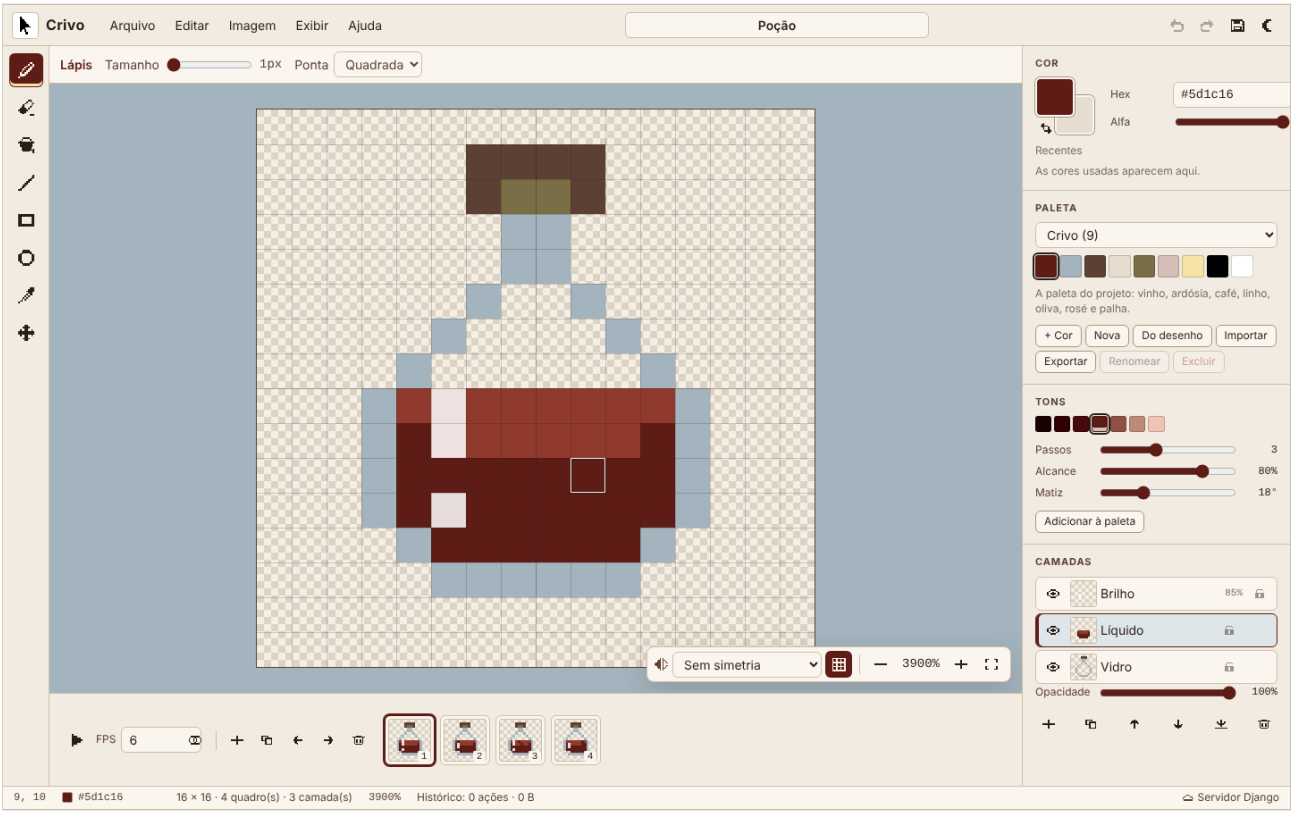
```
npm run servir
```

O script `scripts/servir.js` é um servidor de arquivos estáticos sem dependências que entrega a pasta `editor/static/editor/` em **http://localhost:5173**. O editor percebe que não há API e passa a salvar projetos no IndexedDB e paletas no `localStorage` do navegador. A barra de status mostra **Modo local (navegador)**.

Qualquer outro servidor estático funciona, inclusive GitHub Pages ou `python -m http.server` dentro de `editor/static/editor/`. Abrir o `index.html` direto do disco não funciona, porque navegadores bloqueiam módulos JavaScript carregados por `file://`.

Capítulo 3

Conhecendo a tela

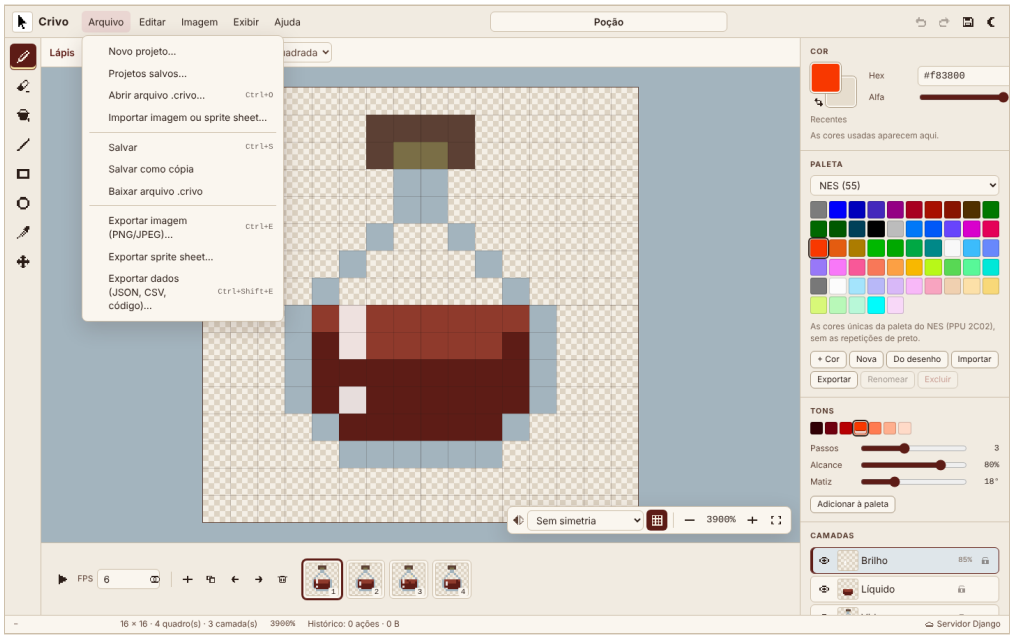


Tela principal com o exemplo da poção: ferramentas à esquerda, opções no topo, matriz no centro, cores e camadas à direita e linha do tempo embaixo.

Área	O que tem
Topo	Logo, menus Arquivo, Editar, Imagem, Exibir e Ajuda, nome do projeto (clique para renomear), desfazer, refazer, salvar e tema
Ferramentas	Lápis, borracha, balde, linha, retângulo, elipse, conta-gotas e mover a visão
Barra de opções	Opções da ferramenta ativa: tamanho e ponta do pincel, modo da borracha, vizinhos e tolerância do balde, preenchimento das formas
Visor	A matriz ampliada, com fundo quadriculado na transparência, grade, eixos de simetria e o contorno do pincel sob o cursor
Controles da visão	No canto do visor: simetria, grade, zoom e ajustar à tela
Painel lateral	Cor (principal, secundária, hexadecimal, alfa e recentes), paleta, gerador de tons e camadas
Linha do tempo	Tocar, FPS, papel cebola, ações dos quadros e as miniaturas de cada quadro
Barra de status	Posição do cursor, cor sob o cursor, tamanho do projeto, zoom, tamanho do histórico e onde os dados ficam salvos

Menus

Menu	Comandos
Arquivo	Novo projeto, projetos salvos, abrir arquivo .crivo, importar imagem ou sprite sheet, salvar, salvar como cópia, baixar .crivo, exportar imagem, sprite sheet e dados
Editar	Desfazer, refazer, limpar camada, espelhar camada na horizontal e na vertical
Imagem	Tamanho da matriz, girar 90° à direita e à esquerda
Exibir	Grade, papel cebola, aproximar, afastar, ajustar à tela, tamanho real e tema claro ou escuro
Ajuda	Atalhos do teclado e sobre o Crivo



O menu Arquivo, com os atalhos de cada comando.

Capítulo 4

Ferramentas de desenho

O botão esquerdo desenha com a **cor principal** e o direito com a **cor secundária**. Toda ferramenta respeita a simetria ativa e só altera a camada selecionada. Se ela estiver oculta ou bloqueada, o Crivo avisa e não desenha.

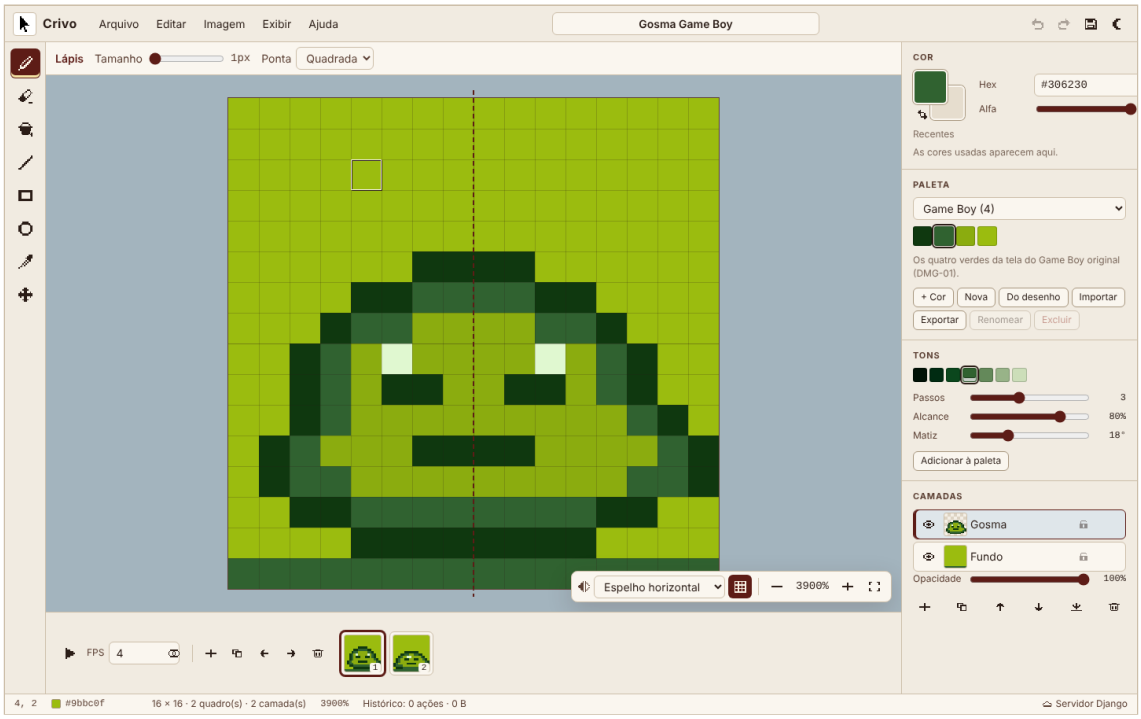
Ferramenta	Atalho	Como funciona	Opções
Lápis	B	Pinta pixel a pixel. Entre dois pontos do mouse, a linha de Bresenham preenche os buracos. Shift + clique traça uma reta desde o último ponto.	Tamanho (1 a 16), ponta quadrada ou redonda
Borracha	E	Igual ao lápis, mas deixa o pixel transparente ou pinta com a cor secundária.	Tamanho, ponta, apagar para transparente ou cor secundária
Balde de tinta	F	Preenche a área de cor parecida em volta do clique.	4 ou 8 vizinhos, tolerância (0 a 100%), só área contígua, ler da camada ou de todas
Linha	L	Arraste de uma ponta à outra. Shift trava em 0°, 45° e 90°.	Tamanho do pincel
Retângulo	R	Arraste de um canto ao outro. Shift faz um quadrado, Alt desenha a partir do centro.	Preenchido
Elipse e círculo	O	Arraste a caixa da elipse. Shift faz um círculo, Alt desenha a partir do centro.	Preenchido
Conta-gotas	I	Clique para pegar a cor do pixel. Arrastar continua pegando.	Ler da camada ou de todas
Mover a visão	H	Arraste para mover a matriz pela tela.	—

> Durante o arraste das formas, a barra de status mostra a largura e a altura da forma em pixels. Esc cancela a forma ou o traço em andamento.

Simetria

Os modos **espelho horizontal** (esquerda e direita), **espelho vertical** (cima e baixo) e **horizontal e vertical** (quatro quadrantes) espelham cada pixel pintado em torno do centro da matriz. Num desenho de largura L, o pixel da coluna x ganha um par na coluna L − 1 − x. Os eixos aparecem tracejados no visor e o contorno do pincel aparece também nas posições espelhadas.

A simetria vale para o lápis, a borracha, as formas e o balde (que preenche também a partir do ponto espelhado). O atalho M alterna entre os modos.



Simetria horizontal: o eixo tracejado divide a matriz e tudo o que é desenhado de um lado aparece do outro.

Capítulo 5

Visualização

Ação	Como fazer
Aproximar e afastar	Roda do mouse (o ponto sob o cursor fica parado), botões − e +, ou as teclas `+` e `-`
Mover a visão	Segure `Espaço` e arraste, use o botão do meio do mouse, a ferramenta Mover, Shift + roda (horizontal) ou o arraste lateral do touchpad
Ajustar à tela	Tecla `0` ou o botão de ajuste: escolhe o maior zoom inteiro em que a matriz cabe inteira
Tamanho real	Tecla `1`: cada pixel da matriz vira um pixel da tela
Grade	Tecla `G`. Aparece a partir de 400% de zoom, com uma linha mais forte a cada 8 pixels

O zoom é sempre um número inteiro (de 100% a 12800%), para que cada pixel da matriz ocupe um quadrado exato de pixels da tela. O desenho é feito com `imageSmoothingEnabled = false`, que copia os pixels por vizinho mais próximo, sem borrar.

Capítulo 6

Cores e paletas

Cor principal e secundária

O quadrado da frente é a cor principal (botão esquerdo) e o de trás, a secundária (botão direito). Clique num deles para abrir o seletor de cor do sistema. O campo **Hex** aceita `#5d1c16`, `5d1c16`, a forma curta `#a3b` ou oito dígitos com transparência, como `#5d1c1680`. O controle **Alfa** muda a opacidade da cor principal; pixels semitransparentes se misturam com o que estiver nas camadas de baixo.

A tecla `X` troca as duas cores. As 16 últimas cores usadas ficam em **Recentes**.

Paletas predefinidas

Paleta	Cores	Origem
Crivo	9	As sete cores do projeto, mais preto e branco
Game Boy	4	Os verdes da tela do Game Boy original (DMG-01)
NES	55	As cores únicas da PPU 2C02, sem as repetições de preto
Pico-8	16	A paleta fixa do console de fantasia Pico-8
CGA	16	As 16 cores do adaptador CGA da IBM
CGA (modo 4)	4	Preto, ciano, magenta e branco, a combinação mais usada nos jogos CGA
Commodore 64	16	As cores do chip VIC-II
1 bit	2	Preto e branco

Suas paletas

- **Nova** cria uma paleta com as cores principal e secundária.
- **+ Cor** adiciona a cor principal. Numa paleta predefinida, o Crivo oferece criar uma cópia sua antes, porque as predefinidas não mudam.
- **Do desenho** junta todas as cores usadas em todos os quadros, da mais usada para a menos usada, até 256.
- **Ctrl + clique** numa cor remove a cor da paleta. **Renomear** e **Excluir** valem só para as suas paletas.
- Com o servidor, as paletas vão para o banco e aparecem em qualquer navegador. Sem servidor, ficam no navegador atual.

Importar e exportar paletas

Formato	Extensão	Como é
Lospec / HEX	`.hex`	Uma cor por linha, como `5d1c16`

Formato	Extensão	Como é
GIMP, Aseprite, Krita, Inkscape	`.gpl`	Cabeçalho `GIMP Palette`, nome e linhas `R G B nome`
Paint.NET	`.txt`	Uma cor por linha em `AARRGGBB`, com comentários iniciados por `;`
JSON	`.json`	`{"nome": "...", "cores": ["#..."]}` ou só a lista de cores
Texto livre	`.txt`, `.csv`	Qualquer código hexadecimal encontrado no texto

Gerador de tons

O painel **Tons** cria uma rampa de sombras e luzes a partir da cor principal. Em vez de só escurecer e clarear em RGB, ele trabalha no espaço de cor **OKLCH** (luminosidade, croma e matiz), em que passos iguais de luminosidade parecem iguais aos olhos.

Controle	Efeito
Passos	Quantos tons de cada lado (1 a 6). A rampa tem 2 × passos + 1 cores
Alcance	Quanto as pontas da rampa se aproximam do preto e do branco
Matiz	Quantos graus as sombras giram em direção ao azul-violeta e as luzes em direção ao amarelo (o hue shifting da pixel art)

Clique num tom para usá-lo e em **Adicionar à paleta** para guardar a rampa inteira. Cores fora do que a tela consegue mostrar têm o croma reduzido por busca binária até caberem, sem mudar a luminosidade nem o matiz.

Capítulo 7

Camadas e animação

Camadas

A lista mostra a camada mais alta no topo. A camada selecionada recebe o desenho. Cada linha tem o olho (mostrar e ocultar), a miniatura, o nome (clique duas vezes para renomear), a opacidade quando é menor que 100% e o cadeado.

Botão	O que faz
+	Cria uma camada logo acima da selecionada
Duplicar	Copia a camada com o conteúdo de todos os quadros
Subir e descer	Muda a ordem de empilhamento
Mesclar	Junta a camada com a de baixo, aplicando a opacidade dela, e remove a de cima
Lixeira	Exclui a camada (o projeto mantém pelo menos uma)
Opacidade	De 0 a 100%. A mudança entra no histórico quando você solta o controle

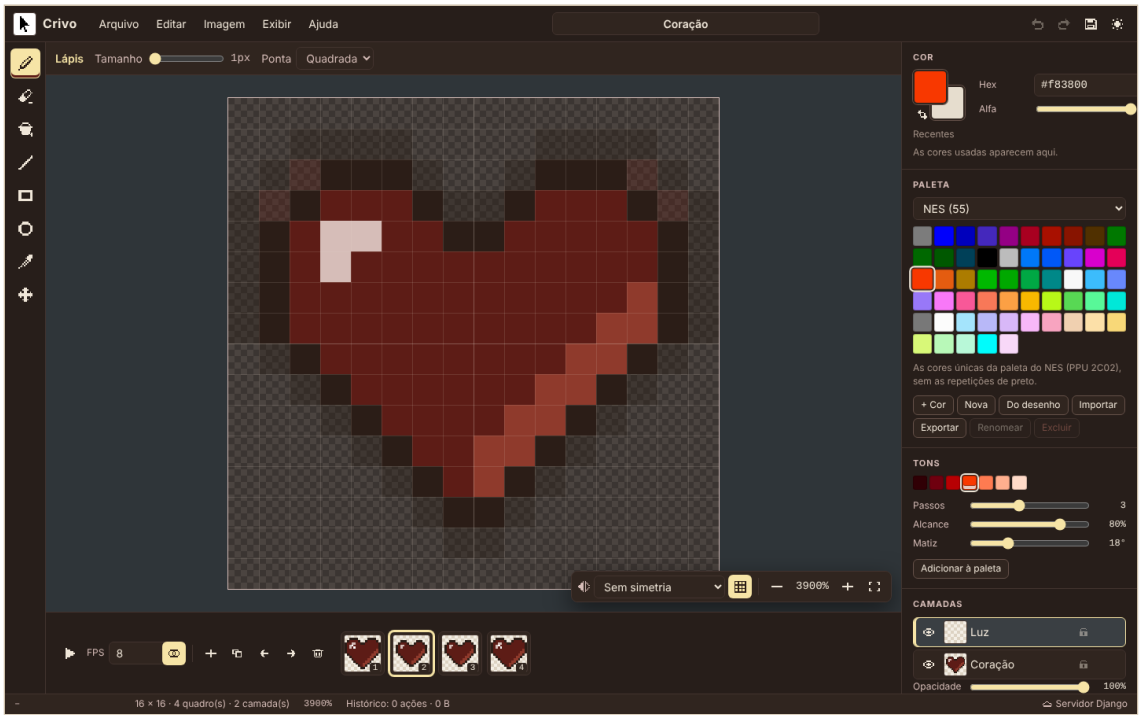
Quadros

Cada quadro tem o seu conteúdo em cada camada; os nomes, a ordem, a visibilidade e a opacidade das camadas valem para todos. Os botões da linha do tempo criam um quadro vazio depois do atual, duplicam, movem para a esquerda ou a direita e excluem.

O campo **FPS** define a velocidade (1 a 60 quadros por segundo). O botão tocar, ou `Enter`, repete a animação; parar volta ao quadro em que você estava. As teclas `,` e `.` andam entre os quadros.

Papel cebola

Com o papel cebola ligado (`K`), o quadro anterior aparece por baixo do atual com 32% de opacidade, para você acompanhar o movimento enquanto desenha. Ele não aparece durante a reprodução nem é exportado.



Tema escuro, com o coração batendo no quadro 2 e o papel cebola mostrando o quadro 1.

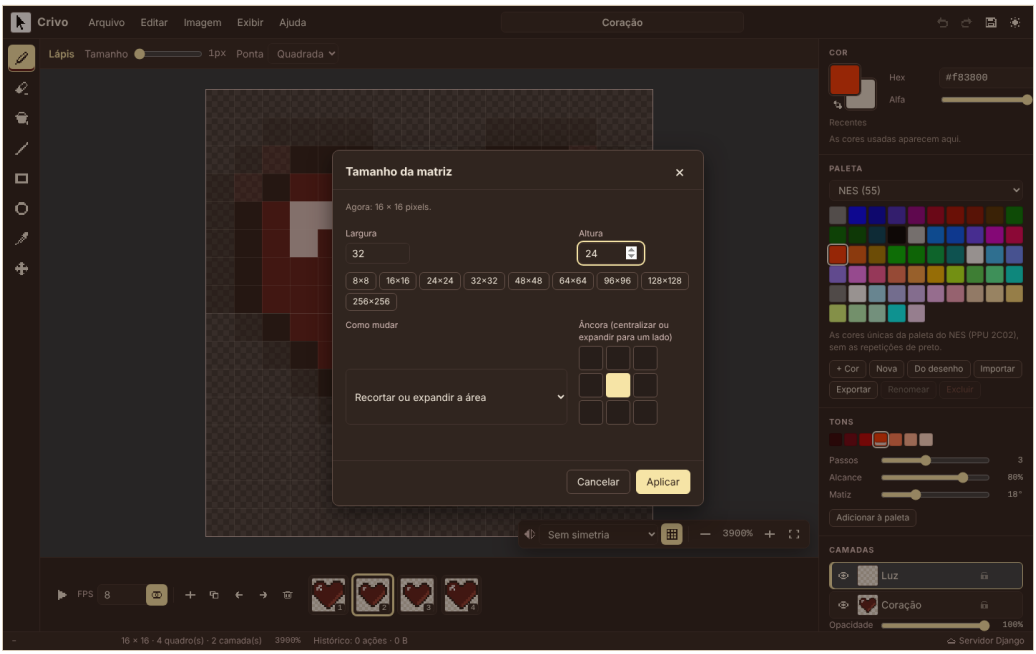
Capítulo 8

Tamanho da matriz e transformações

Em **Imagem → Tamanho da matriz** (`Ctrl` + `Alt` + `C`) você escolhe a nova largura e altura (1 a 1024), com atalhos para os tamanhos mais comuns, e um de dois modos:

Modo	O que acontece
Recortar ou expandir a área	Os pixels não mudam de tamanho. A âncora diz onde o desenho atual fica: no centro, a área cresce ou encolhe igualmente dos dois lados; num canto ou lado, cresce só na direção oposta
Escalar o desenho	O desenho é redimensionado por vizinho mais próximo: cada pixel novo copia o pixel original mais próximo do seu centro, sem criar cores novas

Os dois modos valem para todas as camadas e quadros e podem ser desfeitos. **Girar 90°** troca a largura pela altura; **espelhar** vale só para a camada selecionada no quadro atual.



Diálogo de tamanho da matriz com a âncora no centro.

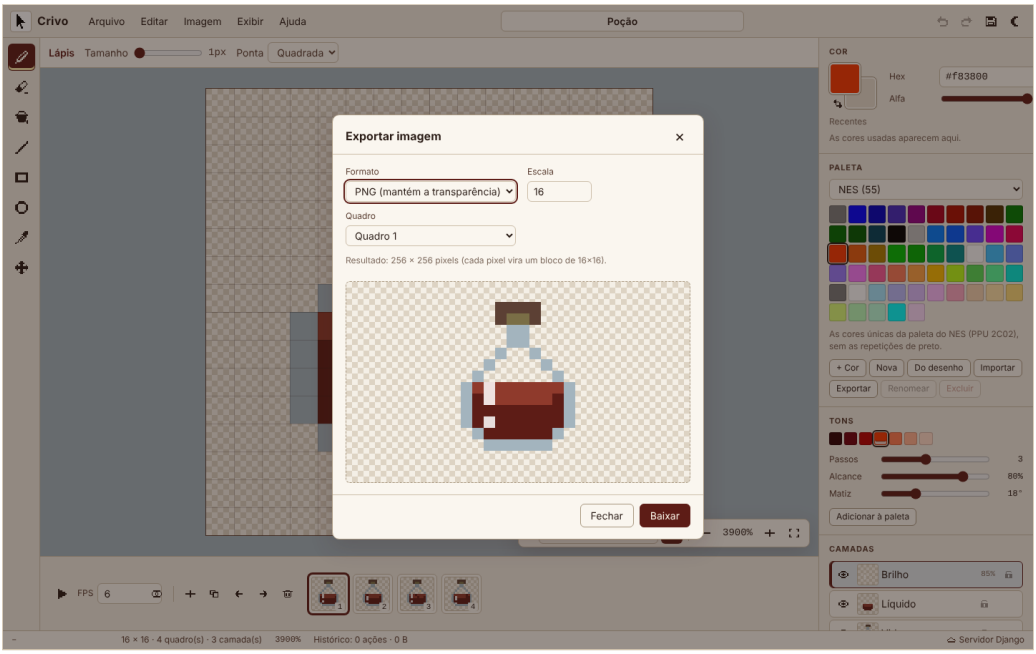
Capítulo 9

Importação e exportação

Exportar imagem

Arquivo → Exportar imagem (`Ctrl` + `E`) gera PNG ou JPEG de um quadro. Com escala 1, o arquivo tem exatamente o tamanho da matriz (pixel art pura). Com escala N, cada pixel vira um bloco de N×N, para não ficar minúsculo em redes sociais ou documentos. A ampliação é sempre por vizinho mais próximo.

O PNG mantém a transparência. O JPEG não tem transparência, então os pixels transparentes recebem a cor de fundo escolhida; a qualidade vai de 10 a 100%.

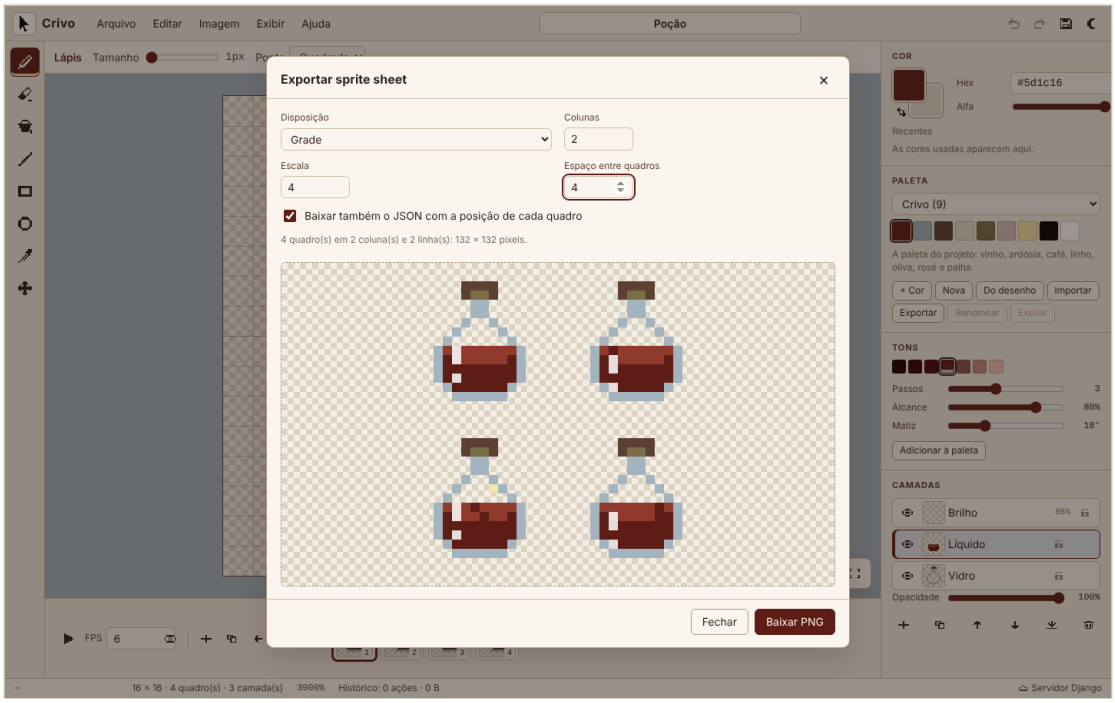


Exportação em PNG com prévia.

Sprite sheet

Arquivo → Exportar sprite sheet junta todos os quadros numa única imagem PNG, em linha (horizontal), coluna (vertical) ou grade, com escala e espaço entre os quadros. Junto vai um JSON com a posição e a duração de cada quadro, no formato que a maioria dos motores de jogo sabe ler:

```
{
  "nome": "coracao-sprites",
  "imagem": "coracao-sprites.png",
  "largura": 64, "altura": 16, "fps": 8,
  "quadros": [
    { "indice": 0, "x": 0, "y": 0, "largura": 16, "altura": 16, "duracao": 240 },
    { "indice": 1, "x": 16, "y": 0, "largura": 16, "altura": 16, "duracao": 125 }
  ]
}
```



Sprite sheet em grade de duas colunas, com escala 4 e espaço de 4 pixels.

Exportar dados da matriz

Arquivo → Exportar dados (`Ctrl` + `Shift` + `E`) transforma a imagem em código. Escolha o formato, os valores e os quadros; a prévia atualiza na hora e os botões copiam ou baixam o arquivo.

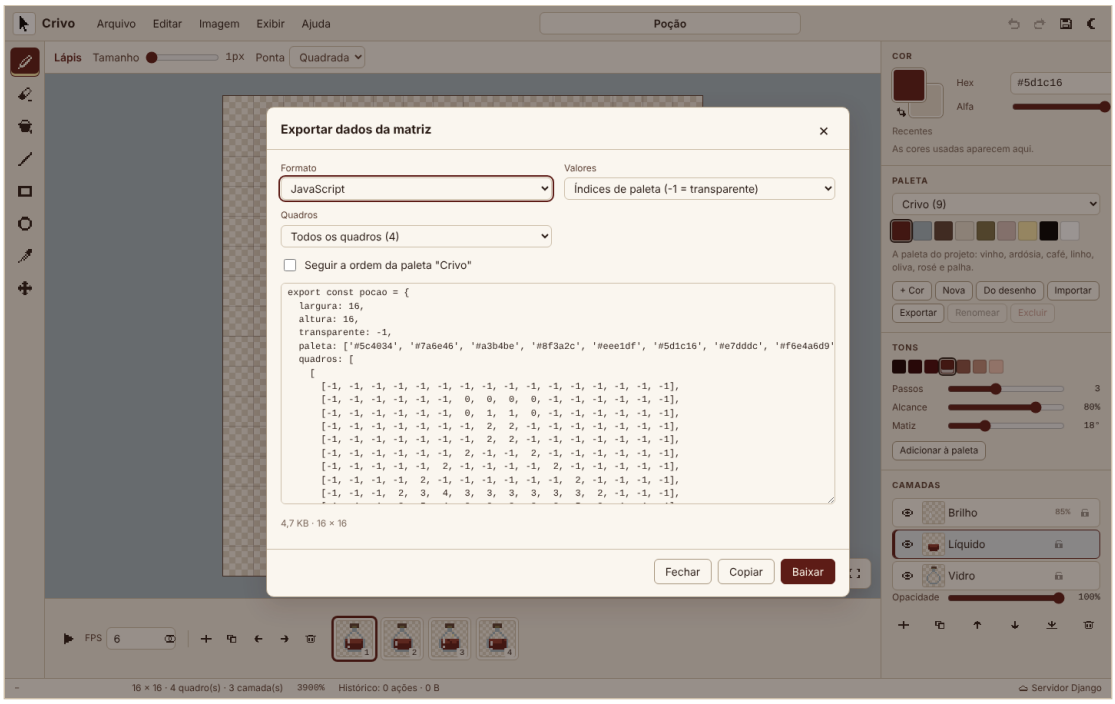
Valores	Como fica cada pixel
Índices de paleta	Um número que aponta para a lista `paleta`; −1 é transparente. Opcionalmente, segue a ordem da paleta atual
Cores em hexadecimal	O código da cor, como `"#5d1c16"`; transparente vira `null` (`None` em Python)

Formato	Arquivo	Estrutura
JSON	`.json`	Objeto com `largura`, `altura`, `paleta` e `quadros[quadro][linha][coluna]`
CSV	`.csv`	Uma linha da matriz por linha do arquivo; quadros separados por uma linha em branco
JavaScript	`.js`	`export const nome = {...}` pronto para `import`
Python	`.py`	`NOME = {...}` com listas aninhadas
C	`.h`	Cabeçalho com `#define` das dimensões, paleta `uint32_t` (RGBA) e matriz `int16_t` ou `uint32_t`
Java	`.java`	Classe final com `LARGURA`, `ALTURA`, `PALETA` (ARGB) e `int[][][] QUADROS`

Um uso típico em JavaScript, desenhando o sprite exportado numa tela do jogo:

```
import { pocao } from './pocao.js';

const ctx = document.querySelector('canvas').getContext('2d');
const quadro = pocao.quadros[0];
quadro.forEach((linha, y) => linha.forEach((indice, x) => {
  if (indice < 0) return;
  ctx.fillStyle = pocao.paleta[indice];
  ctx.fillRect(x * 4, y * 4, 4, 4);
}));
```



Exportação dos quatro quadros da poção como módulo JavaScript.

Importar imagens

Arquivo → Importar imagem ou sprite sheet aceita PNG, JPEG, GIF (primeiro quadro), WebP e BMP de até 4096 pixels de lado. Ao abrir o arquivo, o Crivo procura o **tamanho do pixel**: muitas imagens de pixel art circulam ampliadas, e cada pixel original virou um bloco N×N. Ele mede o comprimento de todas as sequências de cores iguais nas linhas e colunas e calcula o máximo divisor comum delas; se o resultado for 4, cada pixel original ocupa 4×4 e a imagem é reduzida 4 vezes, pegando o pixel do centro de cada bloco.

Abrir como	Resultado
Novo projeto	Um projeto com o tamanho da imagem reduzida
Nova camada	Uma camada nova no quadro atual; o que passar da matriz é cortado
Animação	A imagem é fatiada em quadros do tamanho informado, da esquerda para a direita e de cima para baixo, ignorando os pedaços vazios

O arquivo .crivo

Arquivo → Baixar arquivo .crivo salva o projeto inteiro num JSON legível: nome, tamanho, FPS, paleta, camadas, quadros e o conteúdo de cada célula em RLE codificado em base64. **Abrir arquivo .crivo** (`Ctrl` + `O`) faz o caminho de volta. É o formato que a API recebe e devolve e que a ferramenta em Java lê.

```
{
  "formato": "crivo", "versao": 1, "nome": "Poção",
  "largura": 16, "altura": 16, "fps": 6,
  "paleta": ["#5d1c16", "#a3b4be", "#5c4034"],
  "camadas": [{ "id": "c...", "nome": "Vidro", "visivel": true, "opacidade": 100, "bloqueada":
false }],
  "quadros": [{ "id": "q...", "duracao": null }],
  "celulas": [{ "quadro": "q...", "camada": "c...", "rle": "EAAAAAAEXEBANP8..." }]
}
```

Capítulo 10

Salvando o trabalho

Servidor ou navegador

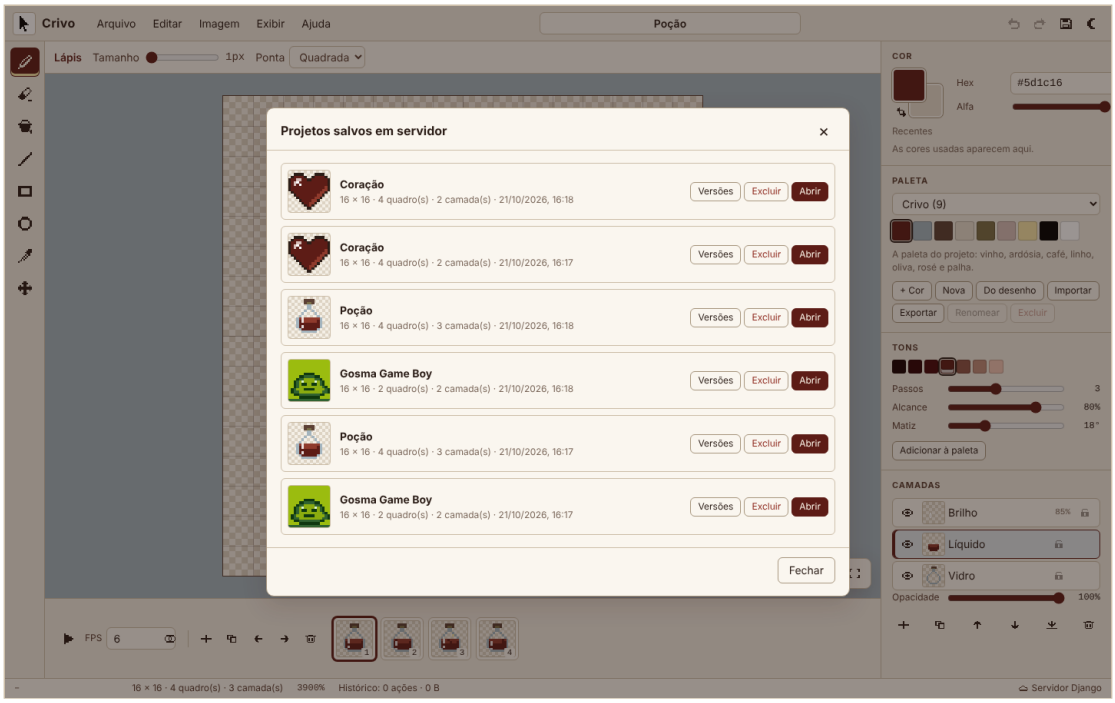
Ao abrir, o editor chama `/api/v1/saude`. Se a API responder, os projetos e as paletas vão para o banco do servidor; se não, ficam no navegador. A barra de status sempre mostra qual dos dois está em uso.

	Servidor Django	Navegador
Onde ficam os projetos	PostgreSQL (ou SQLite)	IndexedDB, banco `crivo`, loja `projetos`
Paletas suas	Tabelas `paleta` e `cor_paleta`	`localStorage`, chave `crivo.paletas`
Versões anteriores	Sim, as últimas 20 por projeto	Não
Abrir em outro computador	Sim, pelo endereço do servidor	Não; use o arquivo .crivo
Rascunho automático	Sim	Sim

Salvar, versões e conflitos

`Ctrl` + `S` salva o projeto atual. A primeira vez cria o projeto; as próximas substituem o conteúdo, e a versão anterior vai para o histórico de versões. **Salvar como cópia** cria um projeto novo. Em **Projetos salvos**, o botão **Versões** lista as versões guardadas e **Restaurar** volta para uma delas (a versão atual também fica guardada, então nada se perde).

Cada projeto tem um número de revisão. Se outra aba ou pessoa salvou o mesmo projeto depois que você o abriu, o servidor responde com conflito e o Crivo pergunta se você quer salvar por cima. A versão da outra pessoa continua no histórico de versões.



Projetos salvos no servidor, com miniatura, tamanho, quadros e camadas.

Rascunho automático

Pouco mais de um segundo depois de cada mudança, e também ao fechar a aba, o projeto aberto é copiado para o IndexedDB. Na próxima vez que o Crivo abrir naquele navegador, o rascunho volta exatamente como estava, inclusive com a marca de mudanças não salvas (o ponto antes do nome na aba).

Capítulo 11

Por dentro: matemática e estruturas de dados

A matriz de pixels

Cada célula (um quadro de uma camada) é um `Uint32Array` com largura × altura posições. O pixel (x, y) fica no índice `y × largura + x`. Cada inteiro de 32 bits guarda os quatro canais; em máquinas little-endian, que são praticamente todas, os bytes na memória ficam na ordem R, G, B, A, a mesma do `ImageData.data`. Por isso a composição final é escrita direto num `Uint32Array` que enxerga o buffer do `ImageData`, sem conversão.

```
cor = (A << 24) | (B << 16) | (G << 8) | R
```

Composição das camadas

As camadas visíveis são combinadas de baixo para cima com o operador "over" em alfa direto, multiplicando o alfa de cada pixel pela opacidade da camada:

```
αs = α_frente × opacidade        αo = αs + α_fundo × (1 − αs)

C = (C_frente × αs + C_fundo × α_fundo × (1 − αs)) / αo
```

Pixels totalmente opacos numa camada a 100% são copiados direto e pixels transparentes são pulados, então o caso comum da pixel art custa uma comparação por pixel.

Linhas: algoritmo de Bresenham

Linhas, o traço do lápis entre dois eventos do mouse e as formas com pincel usam a versão inteira de Bresenham: um erro acumulado decide, a cada passo, se a linha anda em x, em y ou nos dois, sem multiplicação nem ponto flutuante. O resultado tem exatamente um pixel por coluna (ou linha) no eixo dominante, sem buracos e sem degraus duplos.

```
dx = |x1 − x0|;  dy = −|y1 − y0|;  erro = dx + dy
repita:
    pinta(x0, y0); se chegou: pare
    e2 = 2 × erro
    se e2 ≥ dy: erro += dy; x0 += sx
    se e2 ≤ dx: erro += dx; y0 += sy
```

Elipses: ponto médio na caixa

A elipse usa a variante de Alois Zingl do algoritmo do ponto médio, que trabalha com a caixa (x0, y0)–(x1, y1) em vez de centro e raios. Isso permite elipses e círculos de diâmetro par, que não têm um pixel central. O algoritmo percorre um quadrante atualizando incrementos de erro inteiros e espelha cada ponto nos outros três. No modo preenchido, cada linha da elipse vira uma faixa horizontal.

Balde de tinta: preenchimento por varredura

A versão ingênua do flood fill empilha os quatro vizinhos de cada pixel e chega a milhões de entradas na pilha. O Crivo usa preenchimento por varredura (scanline): ao tirar uma semente da pilha, ele expande para a esquerda e a direita até a borda, marca a faixa inteira e, nas linhas de cima e de baixo, empilha só uma semente por trecho contínuo que ainda precisa ser pintado. Com 8 vizinhos, o trecho examinado avança um pixel para cada lado, alcançando as diagonais.

Um vetor `Uint8Array` de visitados evita reprocessar pixels. A tolerância compara cada pixel com a cor do ponto clicado pela maior diferença entre os canais; todos os pixels totalmente transparentes contam como a mesma cor.

RLE com varint

Pixel art tem longas sequências de cores iguais, principalmente transparência. Cada célula é guardada como uma lista de pares (quantidade, cor): a quantidade em varint (7 bits por byte, o bit mais alto indica que há mais bytes) e a cor em 4 bytes RGBA. A codificação faz duas passadas: a primeira conta o tamanho exato e a segunda escreve num `Uint8Array` já do tamanho certo.

Célula	Bruta	Em RLE
32×32 vazia	4.096 bytes	6 bytes
16×16 da poção (vidro)	1.024 bytes	225 bytes
256×256 com fundo de uma cor e um desenho no meio	262.144 bytes	alguns KB

Histórico ilimitado

O histórico é uma pilha de passado e uma de futuro. O que cada entrada guarda depende do tamanho da mudança, para gastar o mínimo de memória:

- **Traço:** durante o arraste, um `Map` guarda a cor original de cada pixel tocado pela primeira vez. Ao soltar o mouse, só os pixels que realmente mudaram viram três vetores: índices, cores antes e cores depois (12 bytes por pixel).
- **Célula:** se mais de 2.048 pixels mudaram (um balde numa área grande, por exemplo), guardar a célula inteira antes e depois em RLE costuma ser menor que a lista de índices.
- **Estado:** mudanças de estrutura (camadas, quadros, tamanho, giro, opacidade, nome) guardam um instantâneo do documento com todas as células em RLE, e células vazias nem entram.

Desfazer aplica o lado "antes" da entrada e a move para o futuro; refazer faz o contrário; uma ação nova descarta o futuro. Não há limite de passos, e o total ocupado aparece na barra de status.

Cores em OKLab e OKLCH

Para gerar tons, a cor sRGB passa para RGB linear (removendo a curva gama), depois para o espaço LMS dos cones por uma matriz 3×3, tira-se a raiz cúbica de cada canal e uma segunda matriz chega ao OKLab (L, a, b). Em coordenadas polares, OKLCH tem luminosidade L, croma $C = \sqrt{(a^2 + b^2)}$ e matiz $h = \text{atan2}(b, a)$.

```
L_sombra = L − (L − 0,08) × t        L_luz = L + (0,985 − L) × t        t = (i / passos) × alcance
```

Cada passo i também gira o matiz em direção a 275° (sombras) ou 95° (luzes), pelo menor arco, e reduz o croma nas luzes para que elas não fiquem saturadas demais. Se a cor resultante sair da gama sRGB, uma busca binária de 24 passos reduz o croma até ela caber.

Escala por vizinho mais próximo

Para ampliar por um fator inteiro N, cada linha da origem é expandida uma vez (cada pixel vira N pixels iguais, com `fill`) e copiada N vezes com `set`. Para redimensionar para qualquer tamanho, cada pixel de destino (x, y) busca a origem em ⌊(x + 0,5) × L_origem / L_destino⌋, o que mantém a imagem centrada e nunca cria cores novas.

Capítulo 12

Arquitetura do código

Módulos do editor

Arquivo	Responsabilidade
`js/nucleo/cores.js`	Empacotar e desempacotar RGBA, hexadecimal, HSL, OKLab, OKLCH, gerador de tons, sobreposição alfa
`js/nucleo/geometria.js`	Bresenham, retângulo, elipse, pincel, espelhos da simetria, restrição de ângulo, detecção do tamanho do pixel
`js/nucleo/preenchimento.js`	Região do balde por varredura e preenchimento
`js/nucleo/rle.js`	RLE com varint e base64 próprio
`js/nucleo/documento.js`	Documento: camadas, quadros, células, composição, redimensionar, escalar, girar, instantâneos
`js/nucleo/historico.js`	Traço, entradas de pixels, de célula e de estado, desfazer e refazer
`js/nucleo/formato.js`	Serializar e desserializar o `.crivo`
`js/nucleo/exportar.js`	Matrizes, geradores de JSON, CSV, JS, Python, C e Java, layout de sprite sheet, escala
`js/nucleo/paletas.js`	Paletas predefinidas, importação, exportação e extração
`js/ui/editor.js`	Estado do editor, eventos, cores, transações de traço e de estrutura, reprodução
`js/ui/visor.js`	Canvas: composição, zoom, pan, xadrez, grade, eixos, prévia das formas e cursor
`js/ui/ferramentas.js`	Ponteiro, teclado e o comportamento de cada ferramenta
`js/ui/paineis.js`	Cor, paleta, tons, camadas e linha do tempo
`js/ui/dialogos.js`	Novo, tamanho, exportações, importação, projetos, versões, atalhos e sobre
`js/ui/armazenamento.js`	Cliente da API e armazenamento no IndexedDB
`js/main.js`	Monta a tela, os menus, os atalhos, o salvamento e o rascunho

Fluxo de eventos

O `Editor` estende `EventTarget` e avisa as partes da tela com eventos: `pixels` (redesenhar a matriz), `documento` (estrutura mudou, refazer listas), `quadro`, `camada`, `cor`, `estado`, `historico`, `zoom` e `salvo`. Nenhum painel conhece os outros; cada um só escuta os eventos de que precisa.

O visor nunca redesenha mais de uma vez por quadro de vídeo: os pedidos se acumulam até o próximo `requestAnimationFrame`. A composição só é refeita quando os pixels mudam; mover o mouse ou a visão só redesenha a imagem já composta.

Servidor Django

Arquivo	Responsabilidade
`crivo/settings.py`	Configuração por variáveis de ambiente, SQLite ou PostgreSQL, segurança e limites
`editor/models.py`	Projeto, Camada, Quadro, Celula, Revisao, Paleta e CorPaleta
`editor/servico.py`	Validação do `.crivo`, gravação em transação, versões, conflitos, composição
`editor/api.py`	Rotas JSON, tratamento de erros e exportação
`editor/rle.py`	RLE e composição em Python, compatíveis com o JavaScript
`editor/exportacao.py`	JSON, CSV, JavaScript e Python pelo servidor
`editor/seguranca.py`	Token de API como alternativa ao CSRF
`editor/views.py`	Entrega o editor com o `<base>` apontando para os arquivos estáticos

O detalhe das tabelas está no PDF de banco de dados.

Capítulo 13

API

Todas as rotas ficam em `/api/v1`, falam JSON em UTF-8 e respondem erros como `{"erro": "mensagem"}`. `GET /api/v1` lista as rotas.

Autenticação

Leituras são livres. Pedidos que alteram dados (`POST`, `PUT` e `DELETE`) seguem a proteção CSRF do Django: o editor recebe o cookie `csrftoken` ao abrir e o devolve no cabeçalho `X-CSRFToken`. Scripts e outros programas podem, em vez disso, mandar `Authorization: Bearer <token>`, desde que `CRIVO_API_TOKEN` esteja definido no servidor.

Rotas

Método e rota	Corpo ou parâmetros	Resposta
`GET /saude`	—	`{status, versao, banco}`; 503 se o banco não responder
`GET /projetos`	`q` filtra pelo nome	`{projetos: [...]}` com id, nome, tamanho, quadros, camadas, revisão e miniatura
`POST /projetos`	`dados` (.crivo), `miniatura` (PNG em data URL), `nome`	201 com o resumo do projeto
`GET /projetos/{id}`	—	Resumo + `dados` no formato .crivo
`PUT /projetos/{id}`	`dados`, `miniatura`, `revisao`, `nome`	Resumo com a nova revisão; 409 se `revisao` estiver desatualizada
`DELETE /projetos/{id}`	—	204
`GET /projetos/{id}/miniatura`	—	PNG
`GET /projetos/{id}/exportar`	`formato` (json, csv, js, py), `modo` (indices, hex), `quadro` (todos ou N), `baixar`	Texto no formato pedido
`GET /projetos/{id}/revisoes`	—	`{revisao_atual, revisoes: [...]}`
`POST /projetos/{id}/revisoes/{n}/restaurar`	—	Resumo com a nova revisão
`GET /paletas`	`predefinidas=0` esconde as predefinidas	`{paletas: [...]}`
`POST /paletas`	`nome`, `cores` (1 a 256)	201 com a paleta

Método e rota	Corpo ou parâmetros	Resposta
`GET/PUT/DELETE /paletas/{id}`	`nome`, `cores`	Paleta, paleta alterada ou 204; 403 nas predefinidas

Códigos de resposta

Código	Quando
200 / 201 / 204	Deu certo
400	Parâmetro inválido (formato de exportação desconhecido, quadro fora do intervalo)
403	Sem CSRF ou token, ou tentativa de alterar uma paleta predefinida
404	Projeto, paleta ou versão não existe
409	Conflito de revisão ao salvar
422	Conteúdo inválido: JSON quebrado, tamanho fora de 1 a 1024, ids repetidos, RLE que não fecha com o tamanho, cor inválida, miniatura que não é PNG
503	Banco de dados fora do ar

Exemplos

```
TOKEN=seu-token
curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
     -d "{\"dados\": $(cat exemplos/coracao.crivo)}" \
     http://localhost:8080/api/v1/projetos

curl "http://localhost:8080/api/v1/projetos/<id>/exportar?formato=csv&quadro=1"

curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
     -d '{"nome": "Terra", "cores": ["#5c4034", "#7a6e46", "#e6ddce"]}' \
     http://localhost:8080/api/v1/paletas
```

Capítulo 14

Ferramenta em Java

A pasta `ferramentas/java` tem uma linha de comando que só precisa do JDK 21. Ela tem o próprio leitor de JSON, decodifica o RLE, compõe as camadas com a mesma fórmula do editor e grava PNG com `javax.imageio`. Os scripts `crivo.sh` e `crivo.cmd` compilam as classes na primeira vez, em `ferramentas/java/out`.

Comando	O que faz
`info <projeto>`	Nome, tamanho, quadros, FPS, camadas e quantidade de cores
`paleta <projeto>`	Cores usadas, da mais usada para a menos usada, com a contagem de pixels
`png <projeto> <saida.png> [--quadro N] [--escala N]`	Um quadro em PNG, ampliado por vizinho mais próximo
`folha <projeto> <saida.png> [--disposicao ...] [--colunas N] [--espaco N] [--escala N]`	Sprite sheet horizontal, vertical ou em grade
`matriz <projeto> [--quadro N|todos] [--modo indices|argb] [--classe Nome]`	Classe Java com a matriz, impressa na saída padrão
`verificar <projeto>`	Confere se o arquivo é um projeto válido

```
cd ferramentas/java
./crivo.sh info ../../exemplos/pocao.crivo
./crivo.sh folha ../../exemplos/coracao.crivo coracao.png --escala 4 --espaco 2
./crivo.sh matriz ../../exemplos/gosma-game-boy.crivo --classe Gosma > Gosma.java
./testar.sh
```

Capítulo 15

Configuração

O servidor lê tudo de variáveis de ambiente. Com Docker, elas ficam no arquivo `.env`, criado pelos scripts de início a partir do `.env.example`.

Variável	Padrão	O que faz
`CRIVO_PORT`	`8080`	Porta publicada pelo Docker
`CRIVO_DEBUG`	`true` fora do Docker, `false` no Docker	Modo de desenvolvimento do Django
`CRIVO_SECRET_KEY`	chave de desenvolvimento	Chave do Django. Obrigatória no Docker; os scripts geram uma aleatória
`CRIVO_ALLOWED_HOSTS`	`localhost,127.0.0.1`	Nomes pelos quais o servidor é acessado, separados por vírgula
`CRIVO_CSRF_TRUSTED_ORIGINS`	vazio	Origens completas (com `https://`) quando há proxy ou domínio próprio
`CRIVO_HTTPS`	`false`	Liga cookies seguros e confia no `X-Forwarded-Proto` do proxy
`CRIVO_API_TOKEN`	vazio	Token para scripts alterarem dados sem CSRF
`CRIVO_MAX_REVISOES`	`20`	Versões anteriores guardadas por projeto
`CRIVO_TAMANHO_MAXIMO_MB`	`24`	Tamanho máximo do corpo dos pedidos
`CRIVO_WORKERS`	`2`	Processos do Gunicorn
`CRIVO_DB_ENGINE`	`sqlite`	`postgresql` para usar PostgreSQL
`CRIVO_DB_HOST` / `CRIVO_DB_PORT`	`localhost` / `5432`	Endereço do PostgreSQL
`CRIVO_DB_NAME` / `CRIVO_DB_USER` / `CRIVO_DB_PASSWORD`	`crivo` / `crivo` / —	Banco, usuário e senha
`CRIVO_SQLITE_PATH`	`crivo.sqlite3`	Arquivo do SQLite
`CRIVO_TIMEZONE`	`America/Sao_Paulo`	Fuso horário das datas
`CRIVO_LOG_LEVEL`	`INFO`	Detalhe do registro

Antes de publicar na internet

- Use uma `CRIVO_SECRET_KEY` longa e aleatória e nunca a coloque no repositório.
- Coloque o domínio em `CRIVO_ALLOWED_HOSTS` e `https://seu-dominio` em `CRIVO_CSRF_TRUSTED_ORIGINS`.
- Sirva por trás de um proxy com HTTPS e defina `CRIVO_HTTPS=true`.
- O Crivo não tem contas de usuário: qualquer pessoa com acesso ao endereço vê e altera os projetos. Para uso público, proteja o endereço no proxy (autenticação básica, VPN ou rede interna).

- Faça backup do volume do PostgreSQL (veja o PDF de banco de dados).

Capítulo 16

Testes e integração contínua

Conjunto	Comando	O que cobre
Núcleo em JavaScript	`npm test`	Cores e OKLCH, Bresenham, retângulo, elipse, pincel, simetria, detecção de escala, balde, RLE, base64, documento, camadas, redimensionar, histórico, formato, exportações, paletas e os exemplos
API Django	`python manage.py test editor`	RLE compatível com o JavaScript, saúde, paletas, ciclo completo de projeto com versões e conflito, exportação, validação, CSRF e token
Ferramenta Java	`bash ferramentas/java/testar.sh`	RLE, JSON, composição, identificadores, geração de classe e PNG ampliado

O workflow de CI do GitHub Actions roda os três conjuntos a cada push e pull request, com a API testada contra um PostgreSQL 17 de verdade, confere se não faltam migrações, gera os PNGs dos exemplos com a ferramenta em Java e constrói a imagem Docker.

Capítulo 17

Personalizando o Crivo

Uma paleta fixa nova

Acrescente um objeto em `PALETAS_PREDEFINIDAS`, no arquivo `editor/static/editor/js/nucleo/paletas.js`. Para a API listar a mesma paleta, repita em `editor/paletas.py` e rode `python manage.py migrate` num banco novo (ou chame `sincronizar_predefinidas` num shell do Django).

```
{
  slug: 'terra',
  nome: 'Terra',
  descricao: 'Tons de solo para cenários.',
  cores: lista('3b2a20 5c4034 7a6e46 a89a6a e6ddce'),
},
```

Uma ferramenta nova

Toda ferramenta é registrada na lista `FERRAMENTAS` de `js/ui/ferramentas.js` (id, nome, atalho, ícone e opções). O comportamento fica nos métodos `baixar`, `moverPonteiro` e `soltar` do `Controlador`. O padrão é sempre o mesmo: abrir um traço, pintar e concluir, para o histórico funcionar sozinho.

```
const traco = this.editor.novoTraco();
if (!traco) return;
traco.pintar(y * largura + x, cor);
this.editor.emitir('pixels');
this.editor.concluirTraco(traco, 'Minha ferramenta');
```

Um formato de exportação novo

Escreva uma função que recebe o resultado de `montarMatrizes` (largura, altura, modo, paleta e quadros) e devolve texto, registre-a em `FORMATOS_DADOS` e em `gerarDados`, no arquivo `js/nucleo/exportar.js`. O diálogo de exportação passa a oferecer o formato automaticamente.

Ícones, cores e atalhos

Os ícones são bitmaps em `js/ui/icones.js`: cada linha é uma string em que `#` é um pixel aceso. As cores da interface são variáveis CSS no início de `css/crivo.css`, separadas para o tema claro e o escuro. Os atalhos ficam na função `atalhos` de `js/main.js`.

Capítulo 18

Atalhos do teclado

Tecla	Ação
`B, E, F, L, R, O, I, H`	Lápis, borracha, balde, linha, retângulo, elipse, conta-gotas e mover
`Alt + clique`	Pegar a cor com qualquer ferramenta de desenho
`Clique direito`	Desenhar com a cor secundária
`Shift (lápis)`	Reta a partir do último ponto
`Shift (formas)`	45°, quadrado ou círculo perfeito
`Alt (formas)`	Desenhar a partir do centro
`Esc`	Cancelar o traço ou a forma em andamento
`X`	Trocar cor principal e secundária
`[ e ]`	Diminuir e aumentar o pincel
`M`	Alternar a simetria
`G`	Grade
`K`	Papel cebola
`Espaço + arrastar`	Mover a visão
`Roda do mouse`	Zoom no ponto do cursor
`+ e −`	Aproximar e afastar
`0 e 1`	Ajustar à tela e tamanho real
`, e .`	Quadro anterior e próximo
`Enter`	Tocar ou pausar a animação
`Delete`	Limpar a camada no quadro atual
`Ctrl + Z`	Desfazer
`Ctrl + Y ou Ctrl + Shift + Z`	Refazer
`Ctrl + S`	Salvar
`Ctrl + O`	Abrir arquivo .crivo
`Ctrl + E`	Exportar imagem
`Ctrl + Shift + E`	Exportar dados da matriz
`Ctrl + Alt + C`	Tamanho da matriz

Tecla	Ação
?	Lista de atalhos

Capítulo 19

Solução de problemas

Problema	O que fazer
A barra de status mostra Modo local, mas o servidor está ligado	Abra o editor pelo endereço do servidor (http://localhost:8080 ou :8000), não pelo `npm run servir`. Confira `http://localhost:8080/api/v1/saude`
Tela em branco ao abrir o index.html direto do disco	Navegadores bloqueiam módulos por `file://`. Use `npm run servir` ou o servidor Django
Não consigo desenhar	A camada selecionada está oculta ou bloqueada; o aviso no canto diz qual. Confira também se a cor principal não está com alfa 0
O balde pintou a imagem toda	Desligue **Só a área contígua**, diminua a tolerância ou troque **Ler de** para a camada atual
Salvar dá conflito	O projeto foi salvo em outra aba depois que você o abriu. Salve por cima ou abra a versão mais nova; nada se perde, as versões ficam guardadas
Erro 403 ao chamar a API por script	Defina `CRIVO_API_TOKEN` no `.env`, reinicie com `docker compose up -d` e mande `Authorization: Bearer <token>`
Erro 400 Bad Request ao acessar por outro endereço	Inclua o nome ou IP em `CRIVO_ALLOWED_HOSTS`
Imagem importada ficou enorme	Aumente o tamanho do pixel no diálogo de importação; o Crivo sugere o valor detectado
`iniciar.cmd` fecha e não abre o navegador	Abra o Docker Desktop, espere ele ficar pronto e rode de novo. `docker compose logs -f web` mostra o que aconteceu
A porta 8080 já está em uso	Troque `CRIVO_PORT` no `.env` e rode `docker compose up -d` de novo
Quero começar do zero	`docker compose down -v` apaga o banco. No modo local, limpe os dados do site no navegador